



PROYECTO 6: NUBE VERDE – MONITOR DE CONSUMO ENERGÉTICO CLOUD

ENTREGABLE FASE 2



FEBRERO 1, 2026

ESCUELA SUPERIOR DE INNOVACION Y TECNOLOGIA - ESIT
SN6

INDICE

INDICE 1

INTEGRANTES 2

RESUMEN 2

DESARROLLO 3

INFRAESTRUCTURA 3

CODIGO O SCRIPTS 4

CONFIGURACIONES 7

PRUEBAS..... 10

EVIDENCIAS..... 12

CONCLUSION 12

INTEGRANTES

Líder de proyecto Jr. (Marco Antonio Arias Artiga)

- Mantener el backlog (tareas, responsables, fechas).
- Coordinar comunicación y entregas semanales.
- Asegurar que la bitácora esté completa y actualizada.

Ingeniero IoT Simulado Jr. (Rene Mauricio Góchez Chicas)

- Diseñar la lógica de simulación (patrones de consumo, horarios).
- Implementar y mantener el simulador.
- Ajustar parámetros de simulación según necesidades de pruebas.

Ingeniero Cloud Jr. (Luis Alexander Marroquín Reyes)

- Configurar los servicios cloud (endpoint, BD, funciones).
- Gestionar seguridad básica y monitoreo mínimo.
- Documentar arquitectura y configuraciones clave.

Desarrollador Dashboard Jr. (Guillermo Ernesto Diaz Peña)

- Implementar el frontend del dashboard (HTML/CSS/JS o framework).
- Integrar las fuentes de datos y mostrar gráficas comprensibles.
- Mejorar UX y visualizaciones según pruebas.

Analista de Datos / QA / Documentador Jr. (Luis Américo Morán León)

- Definir y validar indicadores energéticos.
- Diseñar y mantener la Matriz de Pruebas.
- Redactar el Informe Final y organizar las evidencias.
- Mantener la bitácora clara y ordenada.

RESUMEN

El trabajo realizado se divide en dos implementaciones tecnológicas que aseguran la disponibilidad de los datos almacenados en la nube:

1. Desarrollo del Simulador

Se desarrolló un script en **Python** encargado de emular el comportamiento de dispositivos físicos.

- **Generación de Datos:** El script crea lecturas simuladas para los puntos de medición definidos.
- **Control Temporal:** Se configuró una frecuencia de envío (2 modalidades, tiempo real mediciones en intervalos de 1 a 60 minutos y la generación histórica para simular 24 horas de datos) para replicar un entorno de monitoreo continuo.
- **Validación Local:** Antes de la integración, se realizaron pruebas de salida en consola para asegurar la integridad de los datos.
- **Gestión de Versiones:** Todo el código fue subido a un repositorio, manteniendo un historial de cambios mediante commits organizados.

2. Infraestructura en la Nube

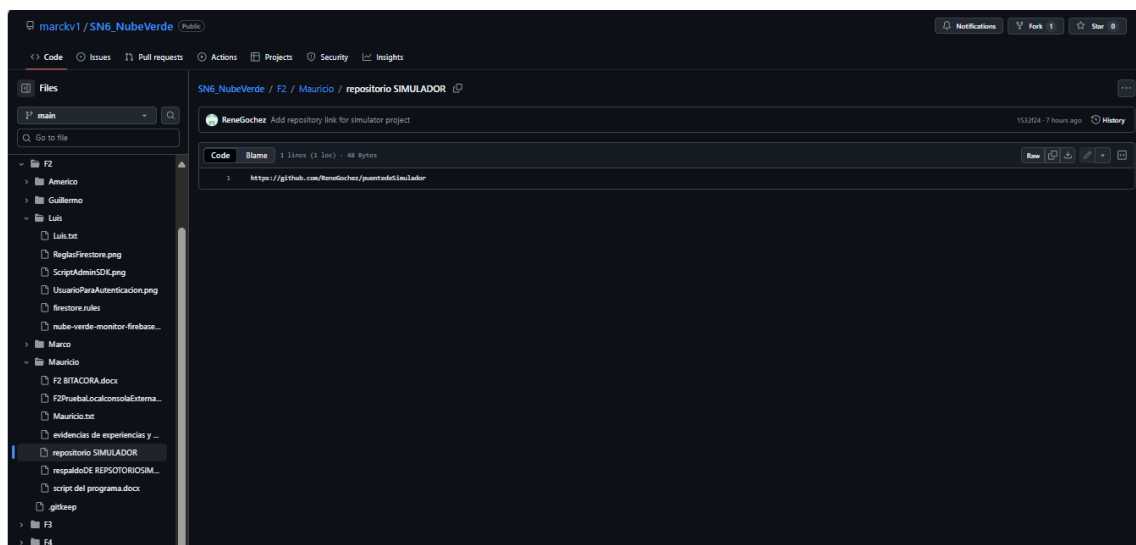
Se configuró **Firebase** como el núcleo del ecosistema cloud para gestionar la llegada de la información.

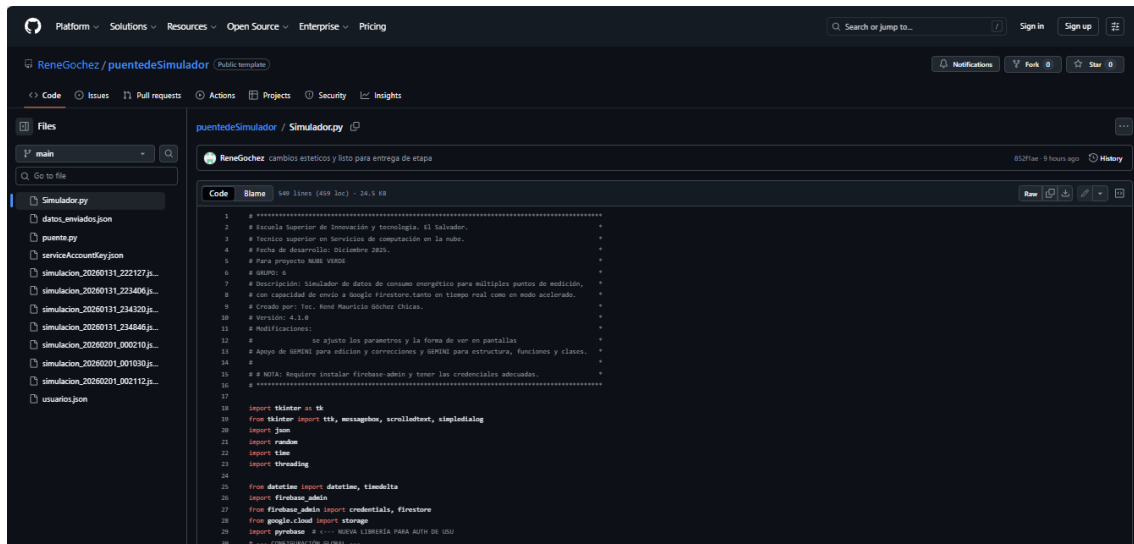
- **Canal de Ingestión:** Se estableció el mecanismo para que el script de Python se comunice con la nube (SDK y script de conexión).
- **Base de Datos NoSQL:** Se implementó el almacenamiento en la nube, asegurando que cada registro incluya:
 - Identificador del punto de medición.
 - Marca de tiempo (Timestamp) precisa.
 - Valor de la lectura.
- **Prueba de Extremo a Extremo:** Se validó exitosamente que un dato generado en el simulador local se refleje correctamente en la base de datos de Firebase sin pérdida de información.

DESARROLLO

INFRAESTRUCTURA

La infraestructura utilizada hasta este punto del proyecto es la de la segmentación de entregables por medio de carpetas divididas en cada fase y en esta fase organización por cada integrante del equipo del proyecto para contener un histórico de archivos subidos, la implementación de un script Python nos otorga la ventaja de no requerir una jerarquía compleja de codificación, implementos externos, librerías o estructura avanzada de carpetas como lo puede contener un proyecto de otro lenguaje o tecnología de programación, se adjunta capturas de la estructura actual, el ingeniero IoT agrego un nuevo repositorio del simulador en su perfil git, se anexa captura y en evidencias el link hacia el mismo.



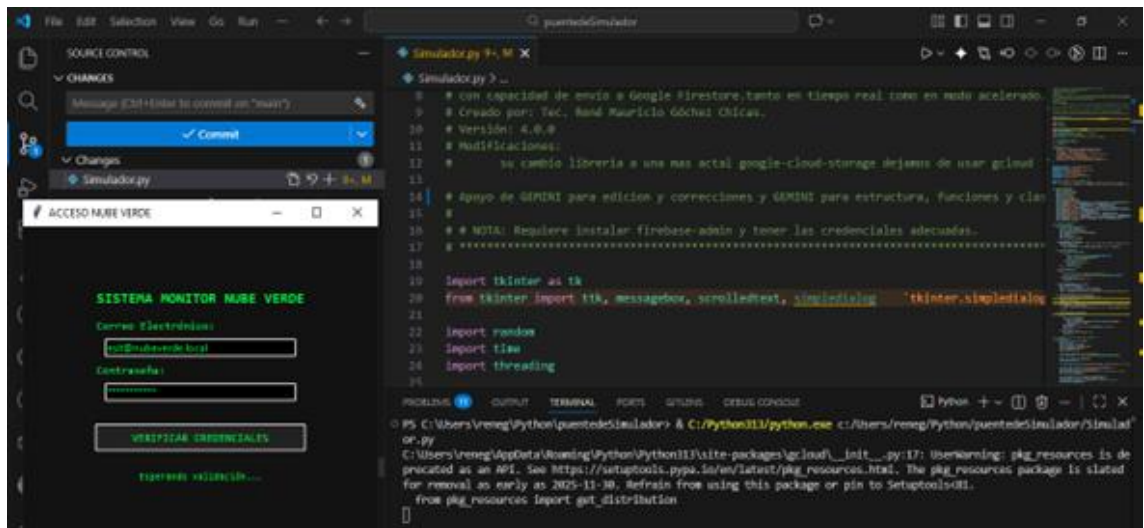


CODIGO O SCRIPTS

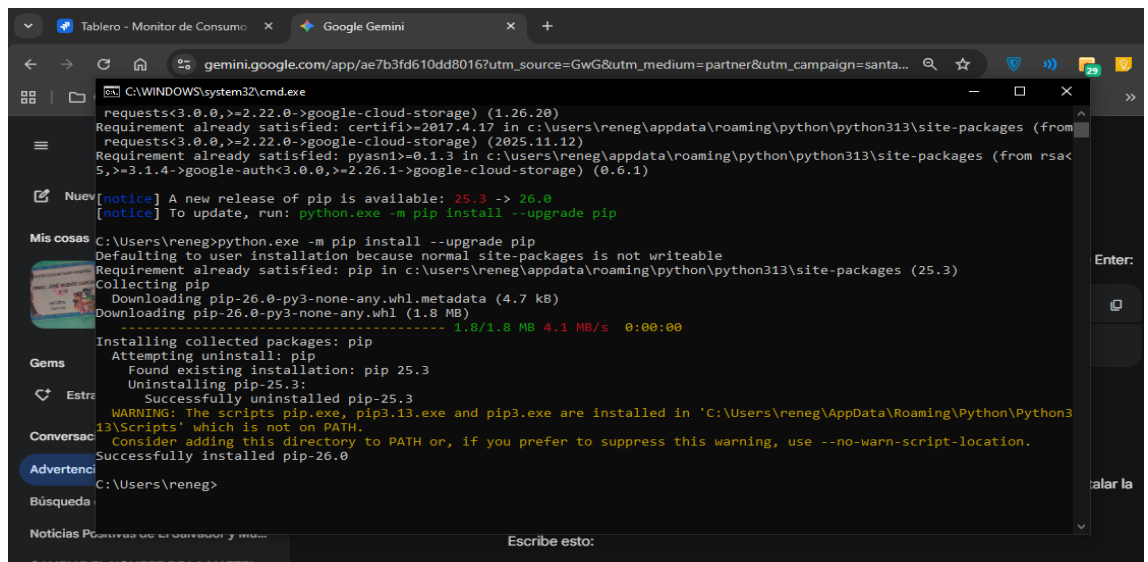
Se muestran partes de la codificación relevantes para la implementación del simulador, conexión a la nube y configuración de la nube:

- Se instala Python en la máquina local y se instalan las librerías de uso de las cuales por medio de scripts de Python se podrán hacer las conexiones a los servicios de Google, en este caso al de firebase concretamente, las librerías son de uso local, por lo que se instalan con comandos, no son archivos o carpetas que se adjunten a la infraestructura ya que son archivos instalados de los servidores de Google y estos se instalan en la máquina donde vaya a funcionar el simulador, se adjuntan scripts ejecutados:

pip install pyrebase4



pip install google-cloud-storage



```
requests<3.0.0,>=2.22.0->google-cloud-storage) (1.26.20)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\reneg\appdata\roaming\python\python313\site-packages (from requests<3.0.0,>=2.22.0->google-cloud-storage) (2025.11.12)
Requirement already satisfied: pyasn1>=0.1.3 in c:\users\reneg\appdata\roaming\python\python313\site-packages (from rsa<5,>=3.1.4->google-auth<3.0.0,>=2.26.1->google-cloud-storage) (0.6.1)
[notice] A new release of pip is available: 25.3 -> 26.0
[notice] To update, run: python.exe -m pip install --upgrade pip
C:\Users\reneg>python.exe -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pip in c:\users\reneg\appdata\roaming\python\python313\site-packages (25.3)
Collecting pip
  Downloading pip-26.0-py3-none-any.whl.metadata (4.7 kB)
  Downloading pip-26.0-py3-none-any.whl (1.8 MB)
----- 1.8/1.8 MB 4.1 MB/s 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.3
    Uninstalling pip-25.3:
      Successfully uninstalled pip-25.3
  WARNING: The scripts pip.exe, pip3.13.exe and pip3.exe are installed in 'C:\Users\reneg\AppData\Roaming\Python\Python313\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
  Successfully installed pip-26.0
C:\Users\reneg>
```

- Conexión de simulador a firebase:

```
25 from datetime import datetime, timedelta
26 import firebase_admin
27 from firebase_admin import credentials, firestore
28 from google.cloud import storage
29 import pyrebase  # <--- NUEVA LIBRERÍA PARA AUTH DE USU
30 # --- CONFIGURACIÓN GLOBAL ---
31 # Configuración del proyecto
32 # --- FIREBASE ---
33
34 FIREBASE_CONFIG = {
35     "apiKey": "AIzaSyAE_pViacv5LR9OpalknySSnuu-TJcTsxw",
36     "authDomain": "nube-verde-monitor.firebaseio.com",
37     "projectId": "nube-verde-monitor",
38     "storageBucket": "nube-verde-monitor.firebaseio.com",
39     "messagingSenderId": "694437356246",
40     "appId": "1:694437356246:web:0b6792fd2a913727739f77",
41     "databaseURL": "https://nube-verde-monitor.firebaseio.com"
42 }
43
44 FIREBASE_CRED_PATH = 'serviceAccountKey.json'
45 COLECCION_FIRESTORE = 'lecturas'
46 BUCKET_NAME = 'nube-verde-monitor.appspot.com'
47 FILE_SENT = 'datos_enviados.json'
48 FILE_UNSENT = 'datos_no_enviados.json'
49 FILE_ACCEL_OUTPUT = 'salida_acelerada.json'
50 FILE_USERS = 'usuarios.json'
```

- Validación con Firebase Auth

```

def check_login(self):
    email = self.user_entry.get()
    password = self.pass_entry.get()

    self.lbl_status.config(text="Conectando con Firebase...", foreground="yellow")
    self.root.update()

    try:
        # Validar con Firebase Auth
        user = self.firebase_auth.sign_in_with_email_and_password(email, password)
        # Si tiene éxito, pasamos a la app principal
        messagebox.showinfo("Éxito", "Token validado correctamente.")
        self.on_success()
    except Exception as e:
        self.lbl_status.config(text="Error de autenticación", foreground="red")
        messagebox.showerror("Acceso Denegado", "Usuario o contraseña de Firebase incorrectos.")

```

- Simular datos, hacer una copia local de los datos y enviar los datos

```

119 def simular_valor(self, pid, current_hour):
120     cfg = self.config[pid][int(current_hour) % 24]
121     if cfg["estado"] == "inactivo": return 0
122     metodo = cfg["metodo"]
123     if metodo == "constante": return float(cfg["constante"])
124     elif metodo == "rango": return round(random.uniform(cfg["min"], cfg["max"]), 2)
125     elif metodo == "probabilistico":
126         return round(random.uniform(0, cfg["max"]), 2) if random.uniform(0, 100) <= cfg["prob"] else 0
127     return 0
128
129 def guardar_en_archivo(self, data_batch):
130     try:
131         self.session_data.extend(data_batch)
132         with open(self.session_file, 'w', encoding='utf-8') as f:
133             json.dump(self.session_data, f, indent=4, ensure_ascii=False)
134         self.log(f"📁 Guardado lote en {self.session_file}")
135         return True
136     except Exception as e:
137         self.log(f"❌ ERROR ARCHIVO: {e}")
138         return False
139
140 def enviar_datos(self, data_batch):
141     if not self.db: return False
142     try:
143         batch = self.db.batch()
144         for item in data_batch:
145             doc_ref = self.db.collection(COLECCION_FIRESTORE).document()
146             batch.set(doc_ref, item)
147         batch.commit()
148         self.log(f"⚡ Enviado lote de {len(data_batch)} registros.")
149         return True
150     except Exception as e:
151         self.log(f"❌ FALLO CONEXIÓN: {e}")
152         return False
153

```

- Muestra de creación de simulación histórica

```

465     def run_accelerated(self):
466         horas = self.horas_aceleradas.get()
467         intervalo = self.intervalo_minutos.get()
468         total_minutos = horas * 60
469         total_pasos = max(1, total_minutos // intervalo)
470
471         self.log_message(f"🚀 Iniciando ráfaga histórica: {horas}h cada {intervalo}min ({total_pasos} lotes)")
472
473         if not self.engine.session_file:
474             timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
475             self.engine.session_file = f"simulacion_{timestamp}.json"
476             self.engine.session_data = []
477
478     def worker():
479         fecha_simulada = datetime.now()
480         for _ in range(total_pasos):
481             batch = []
482             for pid in PUNTOS_ID:
483                 try:
484                     v_min = float(self.individual_configs[pid]["min"].get())
485                     v_max = float(self.individual_configs[pid]["max"].get())
486                 except:
487                     v_min, v_max = 10, 100
488
489                 val = round(random.uniform(v_min, v_max), 2)
490                 batch.append({"id_punto": pid, "consumo_kwh": val, "fecha": self.engine.get_formatted_date(fecha_simulada)})
491
492             if self.dest_var.get() == "DB":
493                 self.engine.enviar_datos(batch)
494                 self.engine.guardar_en_archivo(batch)
495
496                 self.root.after(0, self.update_table, batch)
497                 fecha_simulada += timedelta(minutes=intervalo)
498             self.log_message(f"✅ Simulación acelerada completada. Datos en {self.engine.session_file}")
499
500     threading.Thread(target=worker, daemon=True).start()

```

CONFIGURACIONES

- Paso a paso de creación de un repositorio de prueba (las indicaciones en algunos pasos cambian a medida se avanza el proyecto y los requerimientos, estas instrucciones son generales pero básicas para la creación del mismo), asumiendo que se cuenta con una cuenta en google
 1. Script 1. Creación del Proyecto en la Consola
 2. Ir a Firebase Console en el navegador.
 3. Clic en "Agregar proyecto".
 4. Asigna un nombre (ej. Simulador-IoT-Proyecto).
 5. Desactivar Google Analytics ya que es un entorno de pruebas
 6. Clic en "Crear proyecto".
 7. Configuración de la Base de Datos
 8. Ir a Build (Compilación) > Firestore Database.
 9. Clic en "Crear base de datos".
 10. Seleccionar la ubicación del servidor (ej. us-central1).
 11. Reglas de seguridad: Para el desarrollo inicial, se selecciona "Modo de prueba" (esto permite escritura libre por 30 días).
 12. En producción, se definen reglas que restrinjan el acceso.
 13. Registro de la Aplicación y Credenciales
 14. Para que Python pueda "hablar" con Firebase, necesitamos una llave privada.
 15. Clic en el icono de engranaje (⚙️) junto a "Descripción general del proyecto" > Configuración del proyecto > Cuentas de servicio.
 16. Seleccionar la opción Firebase Admin SDK.
 17. Clic en el botón "Generar nueva clave privada".

18. Se descarga un archivo. json. > serviceAccountKey.json y se colócalo en la carpeta del script de Python.

- Script AdminSDK

```
1 {
2   "type": "service_account",
3   "project_id": "nube-verde-monitor",
4   "private_key_id": "12f...",
5   "private_key": "-----BEGIN PRIVATE KEY-----\nMIIEvQIBAF...",
6   "client_email": "firebase-adminsdk-fb5vc@nube-verde-monitor.iam.gserviceaccount.com",
7   "client_id": "...32235",
8   "auth_uri": "https://accounts.google.com/o/oauth2/auth",
9   "token_uri": "https://oauth2.googleapis.com/token",
10  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
11  "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/firebase-adminsdk-fb5vc@nube-verde-monitor.iam.gserviceaccount.com",
12  "universe_domain": "googleapis.com"
13 }
14
```

- Configuración de usuarios para su autenticación

Authentication

Usuarios Método de acceso Plantillas Uso Configuración Extensions

Las siguientes funciones de autenticación dejarán de estar disponibles cuando se cierre Firebase Dynamic Links pronto: la autenticación a través de vínculos de correo electrónico en apps para dispositivos móviles, así como la compatibilidad con OAuth de Cordova para apps web.

Buscar por dirección de correo electrónico, número de teléfono o UID de usuario **Agregar usuario**

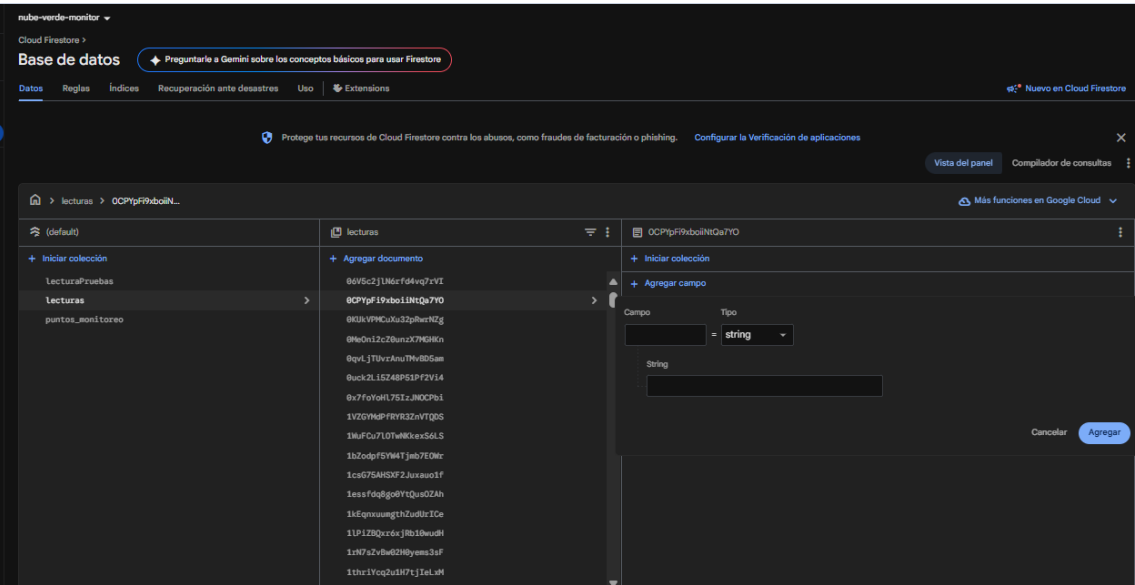
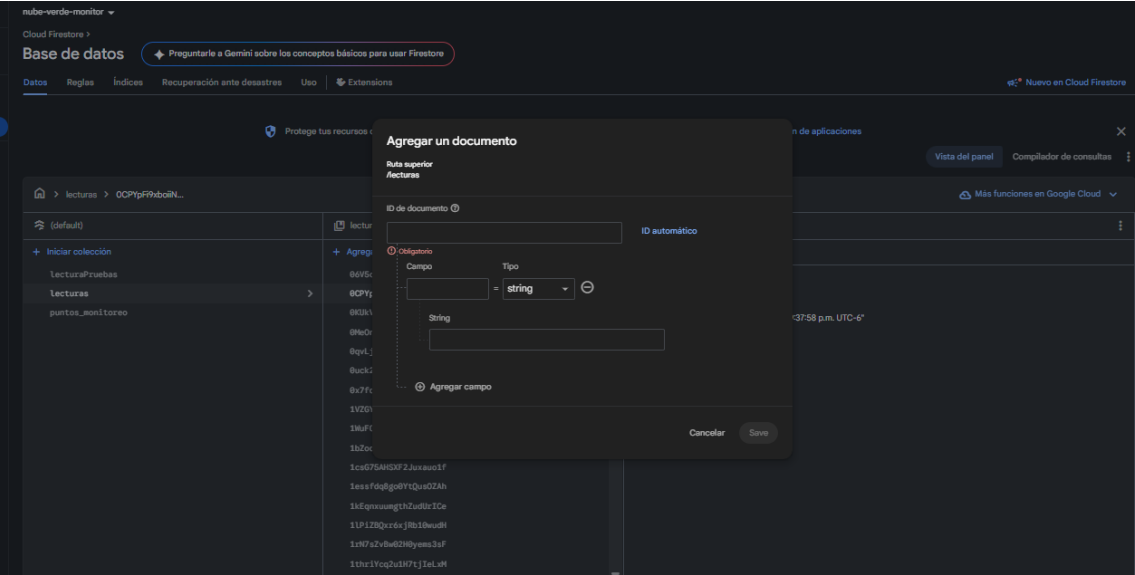
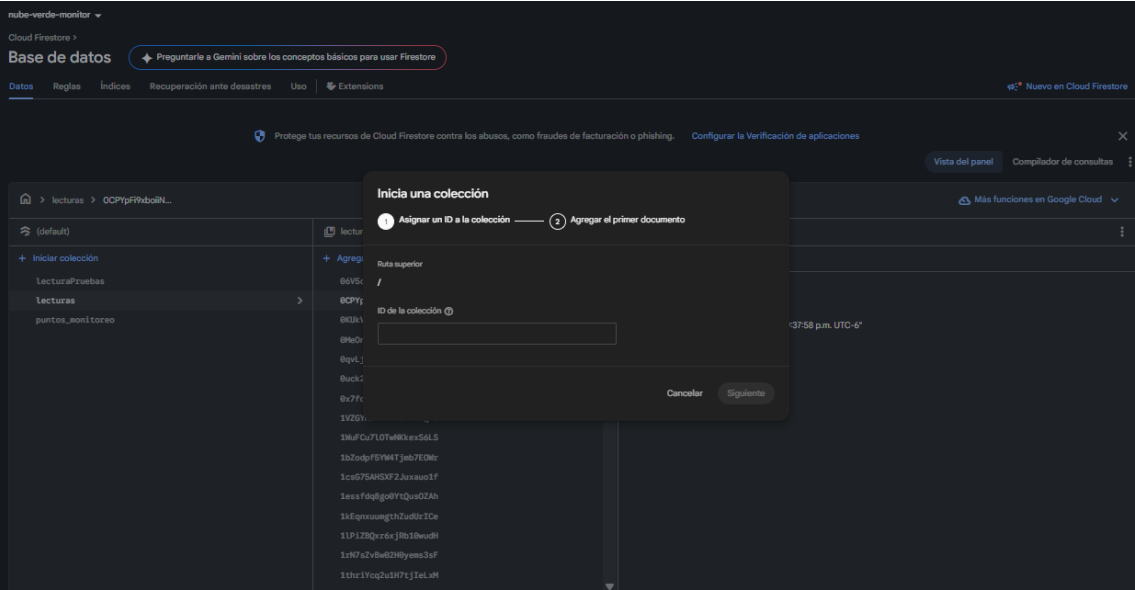
Identificador	Proveedores	Fecha de creación	Fecha de acceso	UID del usuario
esit@nubeverde.local		24 dic 2025	2 ene 2026	jYyWTQvMN6NNDZcCRhhASe...
simulador@nubeverde...		23 dic 2025	16 ene 2026	BXV6SgLKAhTBRNMN7QLKY...

Filas por página: 50 1 - 2 of 2

- Reglas de seguridad para la conexión e interacción del simulador con firebase

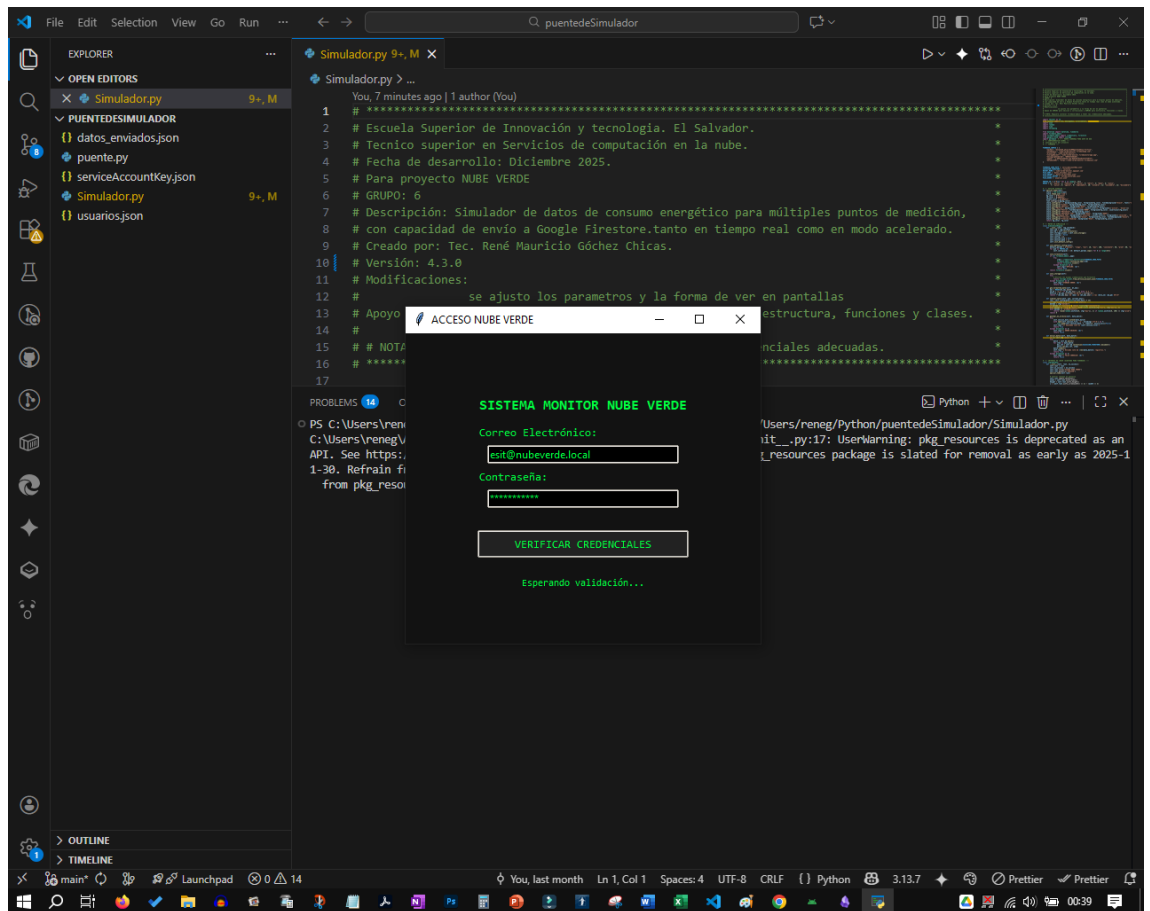
```
Base de datos
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4
5     function isAuthenticated() {
6       return request.auth != null;
7     }
8
9     function isValidLectura(data) {
10      return data.keys().hasAll(['id_punto', 'fecha', 'consumo_kwh', 'estado']) &&
11      data.id_punto is string &&
12      data.id_punto.size() > 0 &&
13      data.consumo_kwh is number &&
14      data.consumo_kwh >= 0 &&
15      data.consumo_kwh < 10000 &&
16      data.estado in ['activo', 'inactivo', 'error'] &&
17      data.fecha is timestamp &&
18      data.fecha <= request.time + duration.value(5, 'm');
19    }
20
21    // =====
22    // COLECCIÓN: puntos_monitoreo
23    // =====
24    match /puntos_monitoreo/{puntoId} {
25
26      // Lectura
27      allow read: if isAuthenticated();
28
29      // Crear: solo admin
30      allow create: if isAuthenticated() && request.auth.token.role == "admin";
31
32      // Update: cualquier usuario autenticado
33      // SOLO puede cambiar el campo "activo" (boolean)
34      allow update: if isAuthenticated()
35      && request.resource.data.diff(resource.data)
36      .changedKeys()
37      .hasOnly(['activo'])
38      && request.resource.data.activo is bool;
39
40      // Eliminar bloqueado
41      allow delete: if false;
42    }
43
44    // =====
45    // COLECCIÓN: lecturas
46    // =====
47    match /lecturas/{lecturaId} {
48
49      allow read: if isAuthenticated() && request.query.limit <= 1000;
```

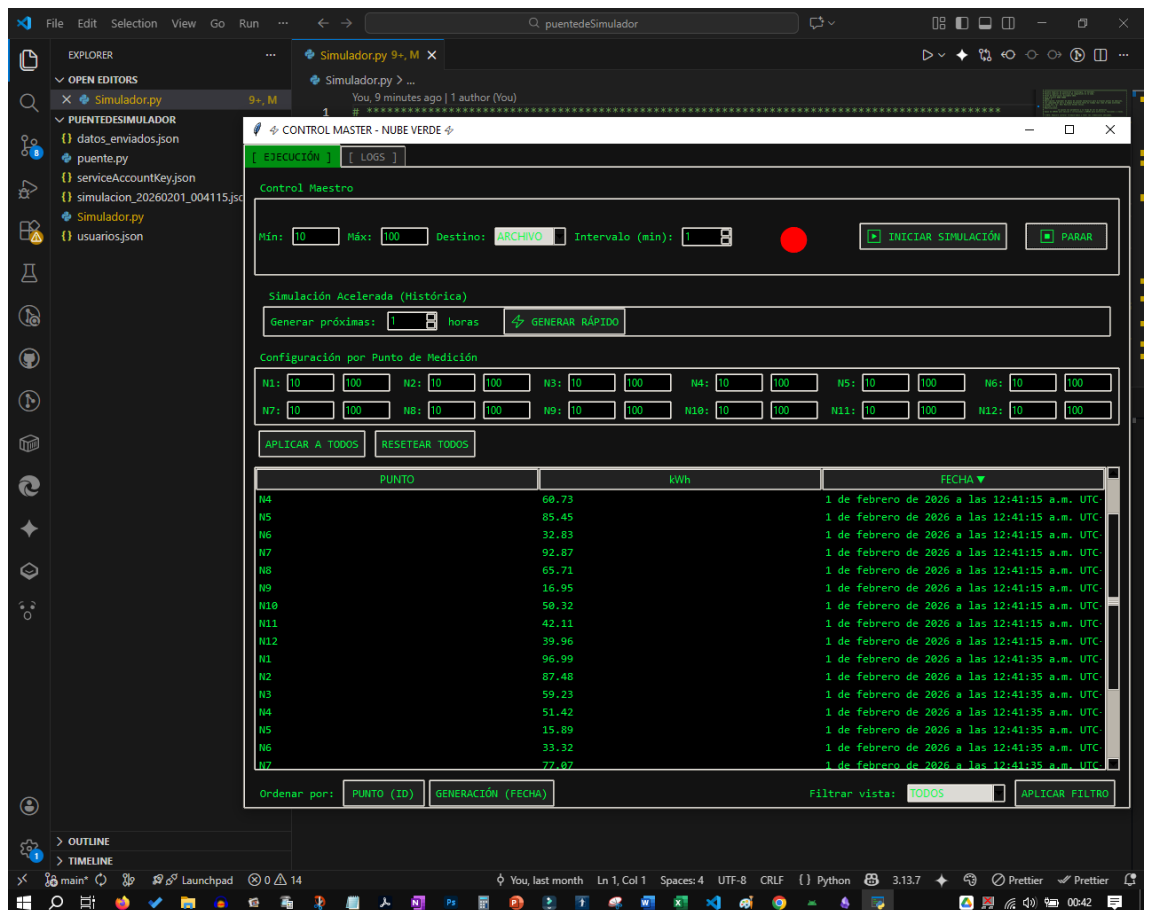
- Creación de colecciones, documentos (registro de información ya sea manual o cargada desde el simulador) y campos para el reposo de datos en la nube de firestore.



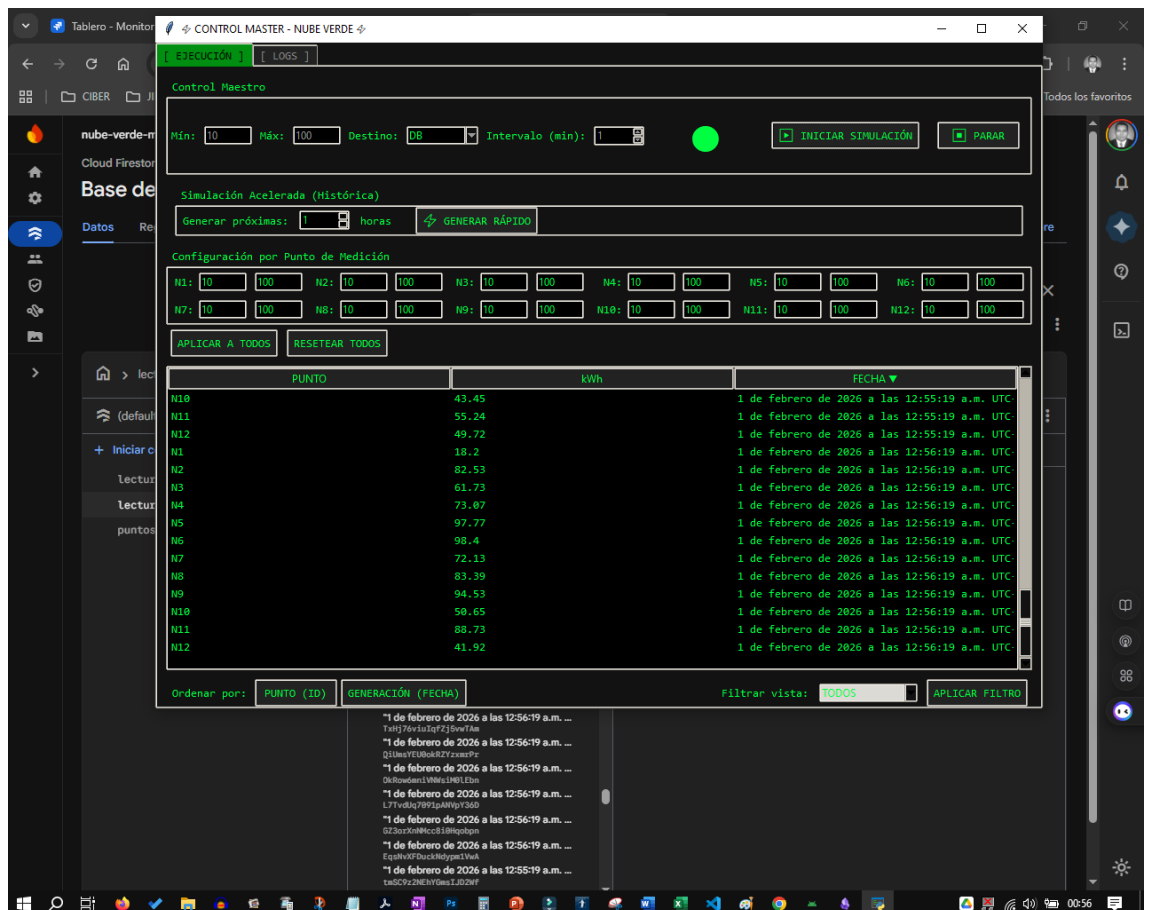
PRUEBAS

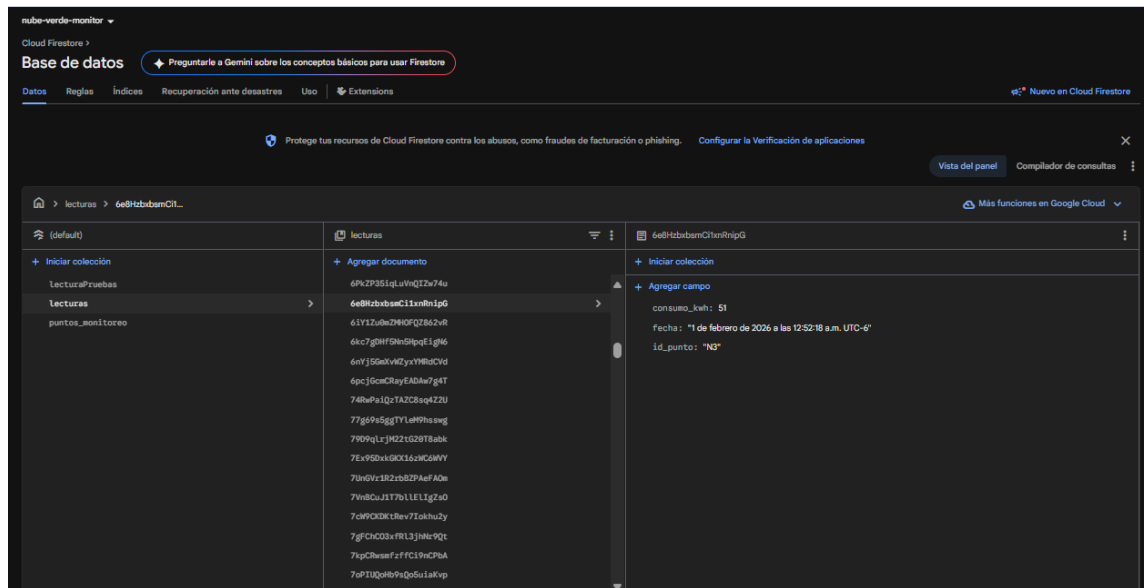
- Local





- Nube





EVIDENCIAS

- Mas detalles en:
[marckv1/SN6_NubeVerde: Repositorio utilizado para el proyecto profesional de servicios en la nube de la entidad educativa ESIT-2025.](#)
- <https://github.com/ReneGochez/puentedeSimulador>
- <https://greenc.atlassian.net/jira/core/projects/MDCEC/board?filter=&groupBy=status&atlOrigin=eyJpIjoiMzRjYTUxNzc0MjA0NDJhYzg5ODAwMDA4Y2U3NDBjMTEiLCJwIjoiajI9>
- Último apartado de bitácoras de este documento.

CONCLUSION

La culminación de esta fase demuestra la viabilidad técnica del proyecto. Se ha logrado transformar una necesidad de monitoreo en un **flujo de datos automatizado y centralizado**.

La elección de Firebase permite que la solución sea altamente escalable y esté lista para la siguiente etapa: la **visualización y análisis de datos**. La arquitectura implementada garantiza que la información sea consistente, esté bien formateada y se encuentre disponible de forma remota, cumpliendo con los estándares de un sistema IoT profesional.

Tareas pendientes para la Fase 3:

- Dashboard inicial
- Indicadores, correcciones y pruebas
- Refinamiento UX y mensaje Nube Verde.